

Sri Lanka Institute of Information Technology



IT3012: Intelligent Agents – Lab 2

Y3.S1.WD.AI.01.01

Malabe Campus

Student ID -	IT24100140
Student Name -	Prathiban HTP
Module Name -	Intelligent Agents
Module Number -	IT3012
Lab Sheet Number -	2
Lab Sheet Date -	2026/July/31

Step 1.1

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{ 'wall_ahead': True/False, 'food_here': True/False }` based on checking the adjacent cell in its current facing direction.

Previous `get_percept` function

```
def get_percept(self) -> dict:
    return {
        'agent_pos': list(self.agent_pos),
        'opponent_positions': [list(op) for op in self.opponents],
        'smells_food': tuple(self.agent_pos) in self.food_positions,
        'hit_wall': tuple(self.agent_pos) in self.walls,
        'collision': self.collision,
        'score': self.score,
        'remaining_food': len(self.food_positions)
    }
```

Updated `get_percept` function

```
43 def get_percept(self) -> dict:
44     dx, dy = {
45         'Up': (0, 1),
46         'Down': (0, -1),
47         'Left': (-1, 0),
48         'Right': (1, 0)
49     }[self.facing_direction]
50
51     ahead_pos = (self.agent_pos[0] + dx, self.agent_pos[1] + dy)
52     wall_ahead = (
53         ahead_pos[0] < 0 or ahead_pos[0] >= self.width or
54         ahead_pos[1] < 0 or ahead_pos[1] >= self.height or
55         ahead_pos in self.walls
56     )
57
```

Step 1.2

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF `food_here` THEN `suck`; IF `wall_ahead` THEN `turn_left`; ELSE `move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

```
agent.py > GreedyGridAgent > sense_and_act
1  # agent.py
2  class SimpleReflexAgent:
3      """A stateless condition-action agent for partial percepts."""
4
5      def sense_and_act(self, percept: dict) -> str:
6          if percept.get('food_here'):
7              return 'Suck'
8          if percept.get('wall_ahead'):
9              return 'Left'
10         return 'Up'
11
12
```

Step 1.3

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
 2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
 3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
 4. Update your IF-THEN rules to query the memory.
Example Logic: IF `wall_ahead` AND `left_is_visited` THEN `turn_right`
 5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.
-

```
16 class ModelBasedAgent:
17     """A condition-action agent that uses memory to avoid repeating loops."""
18
19     def __init__(self):
20         self.percept_history = []
21         self.last_action = None
22         self.wall_escape_index = 0
23         self.relative_pos = (0, 0)
24         self.visited_cells = {self.relative_pos}
25
26     def sense_and_act(self, percept: dict) -> str:
27         self._update_state(percept)
28
29         if percept.get('food_here'):
30             action = 'Suck'
31         elif percept.get('wall_ahead'):
32             escape_actions = ['left', 'Right', 'Down', 'Up']
33             action = escape_actions[self.wall_escape_index % len(escape_actions)]
34             self.wall_escape_index += 1
35         else:
36             self.wall_escape_index = 0
37             action = self._least_visited_move()
38
39         self.last_action = action
40         return action
41
42     def _update_state(self, percept: dict):
43         self.percept_history.append(dict(percept))
44
45         if self.last_action in ('Up', 'Down', 'left', 'Right') and not percept.get('wall_ahead'):
46             x, y = self.relative_pos
47             if self.last_action == 'Up':
48                 y += 1
49             elif self.last_action == 'Down':
50                 y -= 1
51             elif self.last_action == 'left':
52                 x -= 1
53             elif self.last_action == 'Right':
54                 x += 1
55
56             self.relative_pos = (x, y)
57             self.visited_cells.add(self.relative_pos)
58
59     def _least_visited_move(self) -> str:
60         x, y = self.relative_pos
```

Task 2

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

Recall the concept of combinatorial explosion and the size of the percept sequence history...

A Table-Driven Agent must store an action for every possible percept sequence. In complex environments like Chess, the number of percept sequences grows exponentially (combinatorial explosion), making the required table impossibly large. As the agent's lifetime increases, the percept history grows longer, requiring even more memory and computation.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

tb.sliit.fi/mod/resource/view.php?id=476959

PM

IT3012 - Practical 02: Agent Architectures

Explain how your IF-THEN statements map directly to the condition-action rules of the architecture...

The condition-action rules are the if, elif, and else statements. Each rule checks the current percept and immediately selects an action (e.g., IF dirty → THEN suck). The agent does not use memory or percept history.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

Analyze the fatal flaw of reflex agents in environments where critical data is hidden...

The environment was **partially observable**, and the agent had **no percept history or internal memory**. It could only react to the current percept, so it repeatedly executed the same actions (e.g., moving between two clean locations) without realizing the task was already complete.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

Evaluate how your internal memory tracking updates its state before selecting a new rule...

The `ModelBasedAgent` updates its **internal state** using new percepts (sensor model) and predicts how its previous actions changed the environment (transition model). This memory allows it to make better decisions and avoid the infinite loop experienced by the `SimpleReflexAgent`.